For selection by `class`, simply use `[attribute::class="..."]`, and for `id` use `[attribute::id="..."]`. Alternatively, there's an abbreviated form of `attribute::`, which is the `@` character. We can therefore shorten `[attribute::class="..."]` to `[@class="..."]`. And now you understand the very first XPath expression I showed: `'//*[@id="first_para"]'` is the abbreviated form of `/descendant-or-self::*[attribute::id="first_para"]`. Before we go diving into the abbreviations of XPath, let's check out some more predicate features.

A word of warning about `class`: real-world HTML elements can and frequently do have multiple classes, which are assigned as a space-delimited list: `<div class="large green">` means that the `div` element has the classes `large` and `green`. If we use `=` to test for either of these classes, the result is empty:

```
// modify our original XML
$xml = <<<EOS
...
  <documents>
    <git>
      <project class="javascript code">Lovely JS</project>
...
```

Here we've given one of our `project` elements a `class="javascript code"` attribute and removed its `name` attribute. Now if we try to search for one of those class values…

```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query('//project[@class="code"]');
print "Found {$results->length} matching node(s)" . PHP_EOL;
```

```
Output:
Found 0 matching node(s)
```

… we get an empty set. This is straightforward enough: there's no special logic in XPath that recognizes that `class="javascript code"` represents two values. XPath will simply see the entire attribute value of `"javascript code"`, whereas the CSS engine has logic to treat `class` attributes as a space-delimited list. For XPath to do the same, we need to use a function: